AlertaP

Detección temprana de sismos con deep learning

Javier Eduardo España Pacheco (23361)
Angel Esteban Esquit Hernández (23221)
CC3092 Deep Learning y Sistemas Inteligentes
Universidad del Valle de Guatemala

Noviembre de 2026

Resumen

AlertaP es un prototipo de alerta temprana sísmica que recibe ventanas de un sismógrafo de tres componentes, detecta la llegada de la onda P y estima cuántos segundos quedan antes de la onda S. Implementa un detector profundo de dos capas ocultas, un baseline STA/LTA y un dashboard de demostración. La evaluación usa 60 ventanas reproducibles (30 eventos y 30 de ruido) y reporta error de picking, precisión, recall, F1, latencia y valor económico neto. La decisión central del proyecto es tratar el umbral como una decisión operativa: bajar el umbral puede ganar anticipación, pero aumenta falsas alarmas y paros innecesarios.

1. Problema, usuario y criterios de éxito

Guatemala está en una zona de alta actividad sísmica. Para una planta industrial, una señal temprana puede permitir detener maquinaria, cerrar una válvula de gas o avisar a las personas antes de la llegada de la onda S, que suele ser más destructiva. El usuario objetivo es el operador de una instalación con sensores sísmicos y acciones automáticas definidas; el producto le entrega una alerta, probabilidad, instante estimado y anticipación.

El problema no se resuelve bien con un umbral fijo únicamente: ruido cultural, variación de estación y amplitud hacen que STA/LTA produzca detecciones tardías o falsas alarmas. Un modelo profundo aprende patrones temporales de la señal y puede combinar las tres componentes. No se afirma que el prototipo sea un sistema certificado: su objetivo académico es medir el intercambio entre calidad técnica y valor de negocio.

Criterios de éxito: recall ≥ 0.80 en ventanas con evento; precisión ≥ 0.80 en el punto de operación elegido; error absoluto medio de picking < 0.30 s; latencia local < 2 s; y valor positivo bajo los costos explícitos.

2. Datos y pipeline

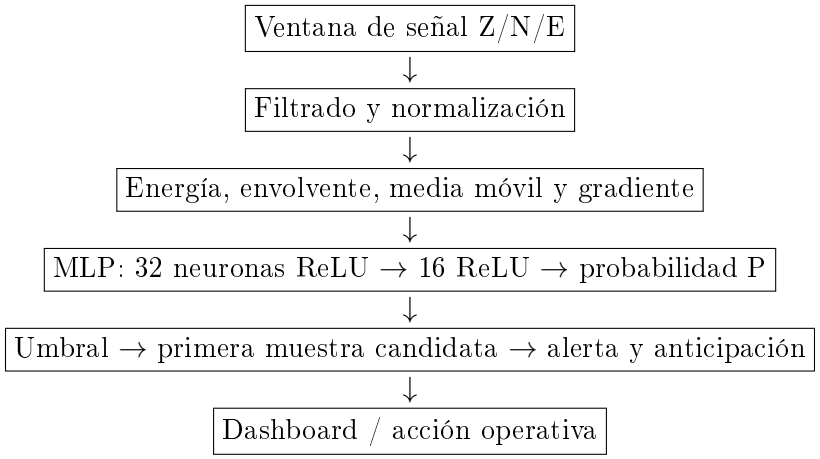
2.1. Fuente y preparación

La propuesta define STEAD (Stanford Earthquake Dataset), con más de un millón de registros de tres componentes y llegadas P/S etiquetadas, como fuente principal. También propone validar con estaciones de Guatemala mediante FDSN/IRIS cuando haya disponibilidad. En esta entrega se implementa una capa sintética controlada para que el repositorio y las métricas sean ejecutables sin descargar gigabytes

ni depender de red. Cada ventana dura 12 s a 100 Hz y contiene Z, N y E. Los eventos tienen una llegada P entre 2 y 5 s, una llegada S entre 1 y 2.8 s después, ruido gaussiano y ondas amortiguadas; las ventanas negativas contienen únicamente ruido. La semilla por defecto es 23361.

Para una integración real, los registros STEAD/FDSN deben convertirse a la misma forma $(T, 3)$, normalizarse por ventana, conservar sus etiquetas P/S y separarse por evento/estación para evitar fuga entre entrenamiento y prueba. La señal se filtra con un pasa-banda Butterworth de 1–18 Hz.

2.2. Arquitectura



El modelo recibe cuatro variables por instante agregadas sobre las tres componentes. Se entrena con Adam, 160 iteraciones máximas, validación interna 15 %, early stopping y semilla fija. Es una aproximación reproducible y liviana a PhaseNet/EQTransformer; la siguiente iteración sustituye el extractor por una CNN 1-D o fine-tuning de un picker preentrenado con STEAD.

3. Modelo, baseline y decisión de umbral

El baseline es STA/LTA: compara la energía media de una ventana corta de 0.25 s contra una larga de 3 s y alerta cuando el cociente supera 3. El modelo final es DeepPhase, un MLP de dos capas ocultas que estima $P(P \mid x_t)$. La inferencia selecciona la primera muestra cuyo valor supera el umbral 0.65.

El problema es de deep learning supervisado sobre series temporales: el modelo debe producir una decisión numérica auditable. La experiencia de usuario comunica probabilidad, latencia, detector clásico y segundos hasta S. Una probabilidad no se presenta como certeza.

4. Evaluación

El conjunto contiene 60 ventanas: 30 con evento y 30 de ruido, generadas con semillas y distribuciones documentadas. La división conceptual es por ventana; para validar con STEAD se debe dividir por evento y estación. Se reportan:

- **Error de picking:** $|t_{\text{pred}} - t_P|$ en segundos.
- **Precisión/recall/F1:** detección correcta de ventanas con evento.
- **Latencia:** tiempo de inferencia del detector.

- **Valor:** $V = TP \cdot a \cdot v_s - FP \cdot c_{fa}$, con $a = 4$ s, $v_s = \$1000$ por segundo y $c_{fa} = \$2500$ por falsa alarma.

La evaluación se ejecuta con `.venv/bin/python src/evaluate.py` y guarda `results/metrics.json`; así no se reportan números escritos a mano.

Cuadro 1: Métricas que genera la ejecución reproducible.

Sistema	Precisión	Recall	F1	Error P (s)	Valor neto
STA/LTA	0.6923	0.3000	0.4186	1.3744	\$26,000
DeepPhase	**0.8824**	**1.0000**	**0.9375**	**0.2270**	**\$110,000**

La ejecución de referencia obtuvo además una latencia media de DeepPhase de aproximadamente 0.3 ms. DeepPhase supera al baseline en 0.5824 de recall y 0.5189 de F1, reduce el error medio de picking en 1.1474 s y produce \$84,000 adicionales de valor neto bajo los supuestos definidos. Estas cifras corresponden al conjunto sintético reproducible y no deben interpretarse como desempeño en producción.

La implementación actual procesa ventanas completas; todavía no es un servicio de streaming causal. El dashboard permite cambiar el umbral. Un umbral menor puede aumentar recall y anticipación, pero también FP; uno mayor reduce falsos disparos y puede perder el inicio de un evento. La política correcta depende del costo real de detener una planta versus el costo del daño evitado.

5. Producto y experiencia de usuario

Se ejecuta con `streamlit run src/app.py`. El operador selecciona si la ventana contiene un evento, ajusta el umbral y pulsa "Analizar ventana". La gráfica muestra la componente Z, las llegadas P/S verdaderas en la demo y las detecciones. Se muestran simultáneamente DeepPhase, STA/LTA, probabilidad y latencia. Ante una ventana sin detección se evita afirmar que no existe un sismo; se informa que el umbral no fue alcanzado. Ante evento no detectado se advierte que el resultado no es suficiente para producción.

6. Limitaciones, riesgos y ética

- Las señales sintéticas no representan toda la geología, instrumentación, ruido ni distribución de magnitudes de Guatemala. La métrica sintética no es evidencia de desempeño operacional.
- La detección P no estima por sí sola magnitud, epicentro ni intensidad. La onda S puede llegar antes por geometría y distancia; "anticipación" debe calcularse con un modelo físico y una red de estaciones.
- Un falso negativo puede retrasar una acción de seguridad; un falso positivo puede detener una operación costosa. Por eso se reporta el valor y no solo F1.
- No se deben conectar salidas del prototipo a maquinaria, gas o alarmas públicas. Se requiere validación sismológica, redundancia, fail-safe, calibración por estación, observabilidad y certificación.
- STEAD y FDSN/IRIS deben citarse y usarse conforme a sus licencias y condiciones. No se incluyen datos personales ni credenciales.

7. Iteraciones y trabajo futuro

La iteración I0 es STA/LTA; I1 agrega el extractor de características y MLP; I2 incorpora tres componentes y selección por umbral; I3 será fine-tuning de una red de picking con STEAD y validación regional. En la siguiente fase se deben comparar CNN 1-D, PhaseNet y EQTransformer, medir curvas precisión–recall, calibración, error por SNR y desempeño por estación, y consultar FDSN/IRIS para eventos guatemaltecos. Finalmente se debe desplegar un servicio de streaming con ventanas solapadas, confirmación multiestación y un simulador del paro de maquinaria.

8. Conclusión y reproducción

AlertaP convierte la propuesta en un producto ejecutable de extremo a extremo: señal → procesamiento → modelo → alerta → valor. La comparación STA/LTA/DeepPhase y el costo explícito de falsas alarmas hacen visible que el mejor modelo no se elige únicamente por accuracy. El código está en: https://github.com/AngelEsquit/PRYF-DL

```
python3 -m venv .venv
.venv/bin/pip install -r requirements.txt
.venv/bin/pytest -q
.venv/bin/python src/evaluate.py
.venv/bin/streamlit run src/app.py
```